4. Podstawowe operacje numeryczne

Operacje elementarne

Zgodnie z określeniami podanymi w poprzednich paragrafach dowolna złożona metoda numeryczna jest skończonym ciągiem elementarnych operacji numerycznych wykonywanych na liczbowych danych wejściowych i wynikach pośrednich w celu otrzymania liczbowego wyniku końcowego; wynik ten jest przybliżonym rozwiązaniem rozpatrywanego problemu matematycznego.

We współczesnych maszynach liczących do zbioru elementarnych operacji numerycznych zaliczamy obok wykonywania działań arytmetycznych również wartościowanie (ewaluację) podstawowych stałych i funkcji matematycznych. W systemach komputerowych operacje zwracające wartość stałych i funkcji są realizowane przez odpowiednie algorytmy numeryczne, najczęściej zaimplementowane w postaci procedur należących do biblioteki standardowej języka programowania (np. w języku C# operacje te są dostępne poprzez klasę biblioteczną o nazwie **Math**). Niektóre operacje ewaluacji nie odwołują się do procedur biblioteki języka i są wykonywane bezpośrednio jako pojedyncze rozkazy procesora matematycznego. Ponadto, operacje ewaluacji stałych mogą sprowadzać się do prostego odczytania wartości tablicowej stałej z pamięci komputera.

Zbiór Ω elementarnych operacji numerycznych dostępnych we współczesnych systemach komputerowych obejmuje następujące elementy:

$$\Omega := \left\{ \begin{array}{l} \mathbf{add}(+), \mathbf{sub}(-), \mathbf{mul}(\cdot, *), \mathbf{div}(/), \mathbf{neg}(-); \\ \mathbf{e}, \mathbf{pi}; \\ \mathbf{abs}, \mathbf{sign}, \mathbf{trunc}, \mathbf{round}(\mathbf{rd}); \\ \mathbf{sqrt}, \mathbf{pow}(\hat{}), \\ \mathbf{exp}, \mathbf{log}, \mathbf{log10}, \\ \mathbf{sin}, \mathbf{cos}, \mathbf{tan}, \mathbf{asin}, \mathbf{acos}, \mathbf{atan}, \\ \mathbf{sinh}, \mathbf{cosh}, \mathbf{tanh}. \end{array} \right\}$$

➤ Nazwy główne wszystkich operacji p/w zbioru przyjmują postać słów kluczowych, czyli zastrzeżonych identyfikatorów. Za niektórymi nazwami głównymi operacji w nawiasach podano nazwę skróconą lub symbol operacji.

➤ Każda operacja *op* zbioru Ω zwraca jako wynik liczbę całkowitą lub rzeczywistą. Większość operacji wymaga podania na wejściu jednego lub dwóch argumentów w postaci liczb całkowitych lub rzeczywistych. Jedynie w przypadku ewaluacji stałych operacje są bezargumentowe. Cel poszczególnych operacji zbioru Ω podano poniżej.

add a, b $(a+b)$
sub a, b $(a-b)$
mul a, b $(a \cdot b)$
div a, b (a/b)

- wykonuje podstawowe działania arytmetyczne na argumentach *a*, *b*, tj. dodawanie, odejmowanie, mnożenie i dzielenie

neg a $(-a)$ - zmienia znak (arytmetyczna negacja) argumentu *a*

e - wartościuje stałą e lub odczytuje wartość 2.71828182845905

pi - wartościuje stałą π lub odczytuje wartość 3.14159265358979

abs a - zwraca wartość bezwzględną argumentu *a*

sign a - zwraca znak argumentu *a*

trunc a - ucina część ułamkową argumentu *a*

round a, n - zaokrągla argument *a* do *n* cyfr znaczących

sqrt a - zwraca pierwiastek kwadratowy argumentu $a \geq 0$

pow a, b $(a \wedge b)$ - zwraca potęgę argumentów *a*, *b* (w niektórych systemach operacja traktowana jako działanie arytmetyczne)

exp a - wartościuje funkcję wykładniczą argumentu *a*

log a - wartościuje funkcję logarytmu naturalnego argumentu $a > 0$

log10 *a* - wartościuje funkcję logarytmu dziesiętnego argumentu $a > 0$

sin *a* - wartościuje funkcję trygonometryczną sinus argumentu *a* w radianach

cos *a* - wartościuje funkcję trygonometryczną kosinus argumentu *a* w radianach

tan *a* - wartościuje funkcję trygonometryczną tangens argumentu *a* w radianach

asin *a* - wartościuje funkcję cyklometryczną arkus sinus argumentu $-1 \le a \le 1$

acos *a* - wartościuje funkcję cyklometryczną arkus kosinus argumentu $-1 \le a \le 1$

atan *a* - wartościuje funkcję cyklometryczną arkus tangens argumentu *a*

sinh *a* - wartościuje funkcję hiperboliczną sinus argumentu *a* w radianach

cosh *a* - wartościuje funkcję hiperboliczną kosinus argumentu *a* w radianach

tanh *a* - wartościuje funkcję hiperboliczną tangens argumentu *a* w radianach

Uwaga: Współczesne środowiska języków programowania udostępniają również podobne operacje wykonywane dla argumentów w postaci liczb zespolonych $z = a + b \cdot i$. Operacje te również mogą mieć odpowiednie wsparcie sprzętowe w systemie komputerowym.

Obliczanie wartości stałych i funkcji

Każda wielkość stała jak i funkcja matematyczna w procesie obliczeń numerycznych musi być wyrażona przez właściwą liczbę. W celu otrzymania wartości liczbowych dla stałych i funkcji, tj. wartościowania (ewaluacji), należy zastosować odpowiednią metodę numeryczną. Dotyczy to również stałych i funkcji należących do zbioru elementarnych operacji numerycznych, który jest zdefiniowany w środowisku maszyny liczącej. Operacje tego zbiory, takie jak ewaluacja stałych **e**, **pi** oraz funkcji **exp**, **log**, **sin**, **cos**, są wykonywane przez maszynę przy użyciu procedur opartych o metody numeryczne.

➢ Wyróżniamy następujące grupy metod obliczania stałych i funkcji:

◆ **metoda ciągu liczbowego** – jeżeli stała jest granicą pewnego ciągu liczb, to jej wartość można obliczyć poprzez obcięcie tego ciągu. Np. stała **e** jest dana ciągiem:

$$x_n := (1+1/n)^n \quad n = 1,2,3,\ldots \quad e := \lim_{n\to\infty} x_n \quad \Rightarrow \quad e \approx x_N$$

gdzie N jest odpowiednio dużą liczbą naturalną, dla której uzyskuje się pożądane przybliżenie.

◆ **metoda szeregu liczbowego** – większość funkcji analitycznych może być zadana poprzez szereg liczb (jest to zwykle szereg Taylora), w takim przypadku ich wartości można obliczyć poprzez obcięcie szeregu. Np. wartości funkcji **exp** , **sin** oraz **cos** dla dowolnych wartości argumentu a dane są szeregami:

$$e^a = 1 + a + \frac{a^2}{2!} + \frac{a^3}{3!} + \ldots \quad \Rightarrow \quad e^a \approx x_N := \sum_{n=0}^{N} \frac{a^n}{n!}$$

$$\sin a = a - \frac{a^3}{3!} + \frac{a^5}{5!} - \frac{a^7}{7!} + \ldots \quad \Rightarrow \quad \sin a \approx x_N := \sum_{n=0}^{N} (-1)^n \frac{a^{2n+1}}{(2n+1)!}$$

$$\cos a = a - \frac{a^2}{2!} + \frac{a^4}{4!} - \frac{a^6}{6!} + \ldots \quad \Rightarrow \quad \cos a \approx x_N := \sum_{n=0}^{N} (-1)^n \frac{a^{2n}}{(2n)!}$$

◆ **metody iteracyjne** – wartość funkcji f dla ustalonego argumentu a można otrzymać z kolejnych przybliżeń y_n na podstawie odpowiedniego schematu iteracyjnego F po podaniu przybliżenia zerowego y_0:

$$y_0 \quad , \quad y_{n+1} = F(y_n) \quad n = 0,1,2,3,\ldots \quad \Rightarrow \quad f(a) \approx y_N$$

Np. wartość funkcji **sqrt** dla danego argumentu a może być obliczona ze schematu wyprowadzonego w przykładzie 1.2:

$$y_0 \quad , \quad y_{n+1} = F(y_n) := 0.5\left(y_n + a/y_n\right) \quad \Rightarrow \quad \sqrt{a} \approx y_N$$

Wyrażenie F tego schematu zostało otrzymane poprzez specyficzne przekształcenia. Formułowanie schematu iteracyjnego może jednak odbywać

się w bardziej systematyczny sposób. W ogólności dla dowolnej funkcji *f* wartość *y* tej funkcji dla ustalonej wartości jej argumentu *x* = *a* można obliczyć na podstawie schematu iteracyjnego stosowanego do rozwiązywania równań nieliniowych. Odpowiednie równanie wyrażone funkcją *F* otrzymujemy jako postać uwikłaną danej funkcji *f* na podstawie jawnego wzoru tej funkcji:

$$y = f(x) \quad \Rightarrow \quad F(x,y) = 0$$

Stosując metodę Newtona (przedstawioną w dalszej części kursu) można po podstawieniu argumentu *a* w miejsce zmiennej *x* wyprowadzić następujący schemat dla tego równania:

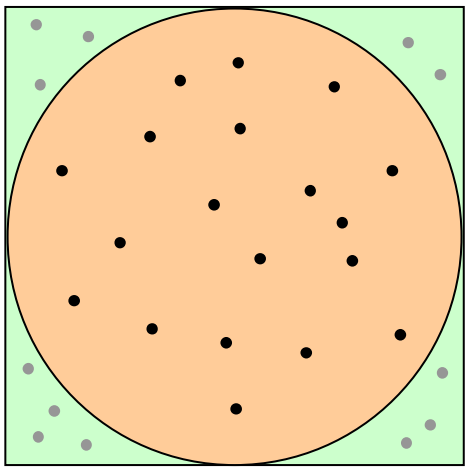
$$y_0 \quad , \quad y_{n+1} = y_n - \frac{F(a,y_n)}{F_y(a,y_n)} \quad n = 0,1,2,3,... \quad \Rightarrow \quad f(a) \approx y_N$$

Zapis F_y oznacza pochodną cząstkową funkcji *F* po zmiennej *y*. Równanie nieliniowe *F*(*x*, *y*) = 0 otrzymujemy poprzez przekształcenie wzoru y = *f*(*x*), tak aby funkcja *f* nie występowała jawnie w tym równaniu. Możemy zatem zaproponować następującą definicję funkcji *F*:

$$y = f(x) \quad \Rightarrow \quad x = g(y) := f^{-1}(y) \quad \Rightarrow \quad F(x,y) := x - g(y)$$

gdzie *g*(*y*) jest funkcją odwrotną do danej funkcji *f*.

- **metody stochastyczne** – używając generatora liczb pseudo-losowych można przeprowadzić odpowiedni ciąg losowań, w którym częstość pojawiania się pewnych wyników jest przybliżeniem wartości rozpatrywanej stałej. Np. wartość stałej **pi** można przybliżyć ¼ częstości pojawiania się losowo wybranych punktów w okręgu wpisanym w kwadrat, przy założeniu, że wszystkie wylosowane punkty mieszczą się w obrębie tego kwadratu:



$$\boldsymbol{\pi} \approx w_N := 4.0 \cdot N_0 \,/\, N$$

gdzie N_0 jest liczbą punktów w okręgu; *N* jest liczbą wszystkich wylosowanych punktów, a więc punktów w kwadracie; przy czym punkty losowe mają rozkład jednostajny.

Przykład 3.1

Jako przykład iteracyjnego obliczania wartości funkcji rozważmy zadanie ewaluacji funkcji proporcjonalności odwrotnej (hiperbola):

$$y = f(x) := 1/x$$

dla ustalonego argumentu $x = b$, przy czym metodę obliczania należy tak sformułować, aby nie występowała w niej operacja dzielenia.
Schemat iteracyjny do obliczania funkcji można wyprowadzić w oparciu o równanie nieliniowe:

$$y = f(x) := 1/x \quad \Rightarrow \quad x = 1/y \quad \Rightarrow \quad F(x,y) := x - 1/y = 0 \quad \Rightarrow$$

$$F_y(x,y) = 1/y^2 \quad \Rightarrow$$

$$x = b \quad y_{n+1} = y_n - \frac{F(b,y_n)}{F_y(b,y_n)} = y_n - \frac{b - 1/y_n}{1/y_n^2} = y_n \cdot (2 - b \cdot y_n) \quad \Rightarrow$$

$$y_0 \quad , \quad y_{n+1} = y_n \cdot (2 - b \cdot y_n) \quad n = 0,1,2,3,... \quad \Rightarrow \; 1/b \approx y_N$$

Wyrażenie w schemacie zbudowane jest wyłącznie z odejmowania i mnożenia. Ponieważ przy właściwie dobranej wartości y_0 początkowego wyrazu ciągu iteracji zachodzi dla kolejnych jego wyrazów: $b \cdot y_n \approx 1$, więc w różnicy p/w wyrażenia nie występuje niekorzystne odejmowanie bliskich sobie liczb, stąd wynik różnicy jak i całego wyrażenia może być otrzymany bez utraty dokładności.
Dla przykładowej wartości argumentu $b = 7$ wartość dokładna rozpatrywanej funkcji dana jest liczbą okresową:

$$1/b = 0.(142857)$$

Aby otrzymać przybliżenie tej wartości z wyżej wyprowadzonego schematu, rozpoczynamy od podania na jego wejściu przybliżenia zerowego, np.:

$$y_0 = 0.1$$

a następnie otrzymujemy kolejne przybliżenia wyższych rzędów:

$$y_1 = y_0 \cdot (2 - b \cdot y_0) = 0.1 \cdot (2 - 7 \cdot 0.1) = 0.13$$
$$y_2 = y_1 \cdot (2 - b \cdot y_1) = 0.13 \cdot (2 - 7 \cdot 0.13) = 0.1417$$
$$y_3 = y_2 \cdot (2 - b \cdot y_2) = 0.1417 \cdot (2 - 7 \cdot 0.1417) = 0.142848$$
$$y_4 = y_3 \cdot (2 - b \cdot y_3) = 0.142848 \cdot (2 - 7 \cdot 0.142848) = 0.142857$$

i tak dalej. Stąd dla kroku $N = 4$ otrzymujemy następujące przybliżenie wartości rozpatrywanej funkcji:

$$1/b \approx y_N = 0.142857$$

co jak widać zgadza się z podaną wartością dokładną w zakresie pierwszych sześciu cyfr mantysy. Wynik p/w jest obarczony błędem:

$$\Delta = |1/b - y_N| = 1.4 \cdot 10^{-7} \approx 2 \cdot 10^{-7}$$
$$\delta = \frac{\Delta}{|y_N|} = \frac{1.4 \cdot 10^{-7}}{0.142857} \approx 1 \cdot 10^{-6}$$

Zatem ostatecznie otrzymujemy przybliżenie ułamka 1/7 z dokładnością do sześciu cyfr znaczących mantysy:

$$1/7 \approx 0.142857 \pm 2 \cdot 10^{-7}$$

Zbieżność p/w metody zależy od wyboru przybliżenia zerowego. Okazuje się, że zbieżność zachodzi, gdy $0 \le y_0 < 2/b$, co łatwo sprawdzić na podstawie analizy wyprowadzonego schematu lub bezpośrednich obliczeń.
Otrzymaną wyżej metodę iteracyjnego obliczania funkcji odwrotności $1/x$ (proporcjonalności odwrotnej) można wykorzystać do emulowania operacji dzielenie **div** w środowisku maszyny liczącej, która nie wspiera sprzętowo tej operacji lub wykonuje ją bardzo wolno w stosunku do operacji mnożenia i odejmowania. Otrzymujemy zatem na podstawie p/w rozważań następującą metodę obliczania wyniku dzielenia danych liczb a, b:

$$\mathbf{div}\ a, b \equiv a/b \equiv a \cdot (1/b) \approx a \cdot y_N$$

Jeżeli założymy, że argumenty a, b są liczbami z przedziału [1, 10) lub są znormalizowane do takiego przedziału (liczby te są zastąpione ich mantysami po odrzuceniu cech), to przybliżenie zerowe dla odwrotności $1/b$ w metodzie iteracyjnej można wybrać jednakowe dla wszystkich wartości argumentu b. Na przykład może to być wartość $y_0 = 0.1$, która spełnia warunek zbieżności metody $0 \le y_0 < 2/b$. Mianowicie, dla $b = 1$ zachodzi: $0 \le 0.1 < 2/b = 2$ oraz dla $b = 10$ mamy: $0 \le 0.1 < 2/b = 0.2$. Podobnie, można przyjąć jednakowy rząd N wynikowego przybliżenia dla wszystkich wartości argumentu b; np. przyjmujemy $N = 4$. Ostatecznie zatem metodę obliczania wyniku dzielenia danych liczb a, b przy podanym opcjonalnie przybliżeniu zerowym y wyniku (domyślnie $y = 0.1$) możemy zrealizować w postaci następującej procedury:

$div(a, b, y = 0.1)$
$\quad y \leftarrow y*(2.0 - b*y)$
$\quad y \leftarrow y*(2.0 - b*y)$
$\quad y \leftarrow y*(2.0 - b*y)$
$\quad y \leftarrow y*(2.0 - b*y)$
return $a*y$

Procedura została zapisana przy użyciu tzw. pseudokodu, w którym zapis postaci ‘ $x \leftarrow w$ ‘ oznacza podstawienie wartości w do zmiennej x, natomiast zapis ‘ **return** w ‘ jest instrukcją powrotu kończącą procedurę i zwracającą jej wynik w do środowiska wywołania procedury.

Uwaga: Dzielenie spośród wszystkich działań arytmetycznych zajmuje najwięcej czasu procesora współczesnych komputerów i prowadzi do wyników najmniej precyzyjnych (np. w koprocesorze 80487 rozkaz dzielenia jest ok. 4.5 razy dłużej wykonywany niż rozkaz mnożenia). Zatem w obliczeniach działanie to powinno być, o ile to możliwe, zastąpione przez bardziej dokładne i mniej czasochłonne operacje. Problem ten zostanie szerzej opisany na wykładzie poświeconym zagadnieniom arytmetyki komputerowej.

Przykład 3.2

Metoda stochastyczna obliczania liczby **pi** posiada jedynie charakter poglądowy, gdyż jest wyjątkowo mało zbieżna. Dla około stu tysięcy losowań uzyskuje się wynik obliczeń z dokładnością zaledwie do trzech cyfr znaczących, 3.14. Na przestrzeni kilku tysięcy lat zaproponowano wiele

bardziej wydajnych metod obliczania tej liczby. Najnowsza metoda opiera się na rozwinięciu liczby **pi** w szereg liczbowy podany w 1997 r. przez trzech matematyków, Bailey - Borwein - Plouffe:

$$\boldsymbol{\pi} \approx x_N := \sum_{n=0}^{N} \frac{1}{16^n}\left(\frac{4}{8n+1} - \frac{2}{8n+4} - \frac{1}{8n+5} - \frac{1}{8n+6}\right)$$

Metoda ta posiada nadzwyczajną zbieżność. Dla szeregu o dziesięciu wyrazach uzyskuje się wynik obliczeń z dokładnością aż do piętnastu cyfr znaczących. Już dla dwóch wyrazów w p/w szeregu wartość ich sumy jest równa liczbie **pi** z dokładnością do czterech cyfr znaczących:

$$x_1 = \sum_{n=0}^{1} \frac{1}{16^n}\left(\frac{4}{8n+1} - \frac{2}{8n+4} - \frac{1}{8n+5} - \frac{1}{8n+6}\right) =$$

$$= \frac{1}{1}\left(\frac{4}{0+1} - \frac{2}{0+4} - \frac{1}{0+5} - \frac{1}{0+6}\right) +$$

$$+ \frac{1}{16}\left(\frac{4}{8+1} - \frac{2}{8+4} - \frac{1}{8+5} - \frac{1}{8+6}\right) =$$

$$= 3.14142 \approx \boldsymbol{\pi} = 3{,}14159265\,358979$$

Metody szybkiego obliczania liczby **pi** umożliwiają obecnie otrzymanie przy pomocy superkomputerów jej wartości przybliżonej z dokładnością do miliona cyfr. Oto początek tej wartości[1]:

```
3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706 79
  82148086513282306647093844609550582231725359408128481117450284102701938521105559644622948954930381 96
  44288109756659334461284756482337867831652712019091456485669234603486104543266482133936072602491412 73
  72458700660631558817488152092096282925409171536436789259036001133053054882046652138414695194151160 94
  33057270365759591953092186117381932611793105118548074462379962749567351885752724891227938183011949 12
  98336733624406566430860213949463952247371907021798609437027705392171762931767523846748184676694051 32
  00056812714526356082778577134275778960917363717872146844090122495343014654958537105079227968925892 35
  42019956112129021960864034418159813629774771309960518707211349999998372978049951059731732816096318 59
  50244594553469083026425223082533446850352619311881710100031378387528865875332083814206171776691473 03
  59825349042875546873115956286388235378759375195778185778053217122680661300192787661119590921642019 89
  38095257201065485863278865936153381827968230301952035301852968995773622599413891249721775283479131 51
  55748572424541506959508295331168617278558890750983817546374649393192550604009277016711390098488240 12
  85836160356370766010471018194295559619894676783744944825537977472684710404753464620804668425906949 12
  93313677028989152104752162056966024058038150193511253382430035587640247496473263914199272604269922 79
  67823547816360093417216412199245863150302861829745557067498385054945885869269956909272107975093029 55
  32116534498720275596023648066549911988183479775356636980742654252786255181841757467289097777279380 00
```

[1] Całość podana jest na stronie http://3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706 79.com.